

18-03-2025

Les outils autour du C

C c

```
1  #include <fcntl.h> // pour le open
2  #include <unistd.h> // pour read
3
4  int main (void) {
5      char data[1024]; //On veut les 1024 octets côte à côte dans la stack
6      int fd = open("hello.md", O_RDONLY);
7      if (fd == -1) {
8          perror("hello.md")
9      }
10     ssize_t nb_bytes = read(fd, data, 1024);
11
12     close(fd);
13
14     write(1, data, nb_bytes)
15
16     return 0;
17 }
```

Au moment du read and write, que se passe-t-il quand il y a des octets nuls ?

Read et Write sont des `syscalls` et les bytes sont userland

La frontière n'est pas clair car le terminal est aussi en C

Le "0" avant RDONLY veut surement dire open

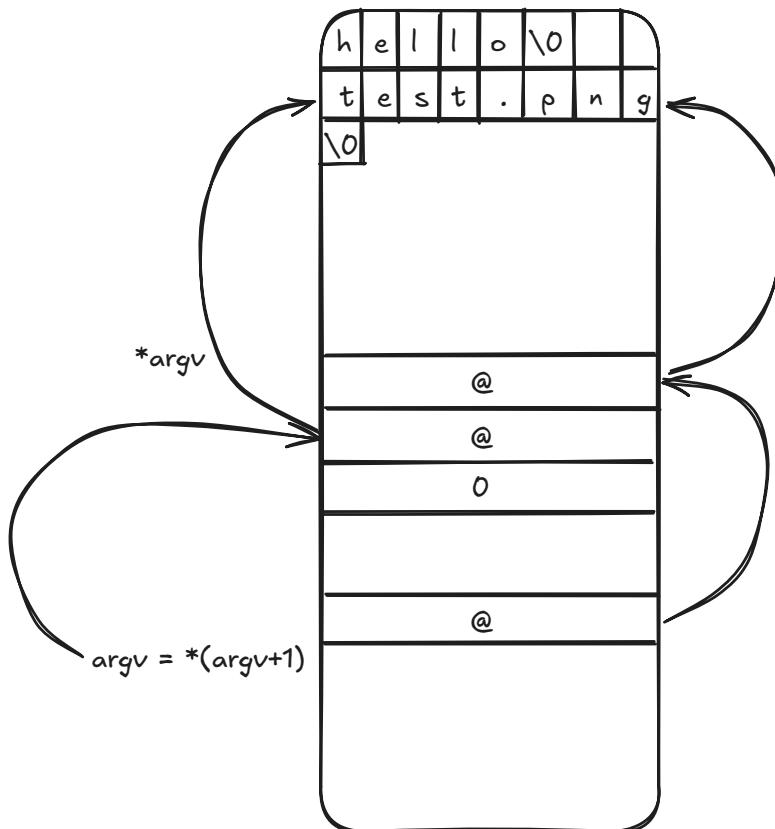
C c

```
1  #include <stdio.h>
2
3  int main (int argc, char **argv) {
4      if (argc < 2) {
5          printf("Please give a filename as argument");
6          return EXIT_FAILURE
7      }
8
9      char data[1024]; //On veut les 1024 octets côte à côte dans la stack
10     int fd = open(*(argv+1), O_RDONLY);
11     if (fd == -1) {
```

```

12     perror("hello.md")
13     return EXIT_FAILURE;
14 }
15 ssize_t nb_bytes = read(fd, data, 1024);
16
17 close(fd);
18
19 write(1, data, nb_bytes)
20
21 return 0;
22 }

```



`readelf -h` → Voir les données de l'entête d'un fichier ELF

`readelf -a` → entrer montrant un interpreter de programme

`objdump -d -Intel ./hello`

`objdump --disassemble=main -Intel ./hello` → Désassemble le code

`nm hello , nm -D /usr/lib/x86_64-linux-gnu/libc.so.6`

`make` → lance le compilateur pour updatet le fichier C so besoin

C c

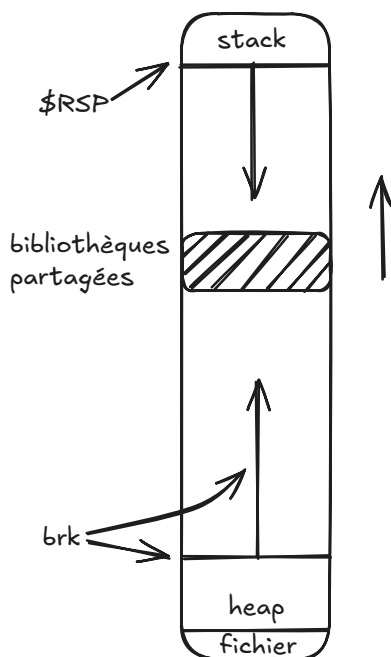
1 //Dans le Makefile

```

2  CFLAGS = -g -std=c99 -O0 -pedantic -pedantic-errors -Wall -Wbad-function-
    cast -Wcast-align=strict -Wcast-qual -Werror -Wextra -Wfloat-equal -
    Wformat=2 -Winit-self -Winline -Wlogical-op -Wnested-externs -Wredundant-
    decls -Wshadow -Wstrict-aliasing -Wstrict-overflow=3 -Wstrict-prototypes -
    Wundef -Wuninitialized -Wwrite-strings -fanalyzer -D_DEFAULT_SOURCE
3
4  hello: hello.c
5      cc $(CFLAGS) hello.c -o hello.o
6
7  hello: hello.o
8      cc hello.o -o hello //pour le lancer en executable
9
10 .PHONY: clean
11 clean:
12     rm -f hello hello.o // supprimer hello et hello.o pour nettoyer le
    projet

```

La heap



Avantage de heap par rapport à la tack

- la stack supprime ses données après le return

C

c

```

1  #include <fcntl.h>
2  #include <unistd.h>
3  #include <stdlib.h>
4  #include <stdio.h>
5
6  char *read_file(char *filename) {

```

```

7     void *base_brk = sbrk(1000); // Increase brk by ak bytes
8     int fd = open(filename, O_RDONLY);
9     if (fd == -1) {
10         perror(filename);
11         return NULL;
12     }
13     ssize_t bytes_read = read(fd, base_brk, 999);
14     close(fd);
15     *((char *)base_brk + bytes_read) = '\0';
16     return (char *)base_brk;
17 }
18
19 int main(void) {
20     if (argc < 2) {
21         printf("Please give a filename.\n");
22         return EXIT_FAILURE
23     }
24     char *file_content = read_file(argv[1]); // argv +1
25     if (file_content){
26         printf("%s\n", file_content);
27         return EXIT_SUCCESS
28     }
29 }
30

```

brk → adresse absolue

sbrk → adresse relatives

void* = malloc(size)

free(void)

C c

```

1     #include <fcntl.h>
2     #include <unistd.h>
3     #include <stdlib.h>
4     #include <stdio.h>
5
6     void print_file_content(char *content){
7         printf("%s\n", content);
8     }
9
10
11    int main(void) {
12        if (argc < 2) {
13            printf("Please give a filename.\n");
14            return EXIT_FAILURE
15        }
16        char *file_content = read_file(argv[1]); // argv +1

```

```
17     if (file_content){
18         print_file_content(file_content);
19         return EXIT_SUCCESS
20     }
21 }
22
```